

Bài: Kiểu dữ liệu Dict trong Python - Phần 1

Xem bài học trên website để ủng hộ Kteam: [Kiểu dữ liệu Dict trong Python - Phần 1](#)

Mọi vấn đề về lỗi website làm ảnh hưởng đến bạn hoặc thắc mắc, mong muốn khóa học mới, nhằm hỗ trợ cải thiện Website. Các bạn vui lòng phản hồi đến Fanpage [How Kteam](#) nhé!

Dẫn nhập

Trong các bài trước, Kteam đã giới thiệu đến bạn [KIỂU DỮ LIỆU SET](#) trong Python

Ở bài này Kteam sẽ đề cập đến các bạn **KIỂU DỮ LIỆU DICT** trong Python. Một trong những kiểu dữ liệu cực kì quan trọng trong Python.

Nội dung

Để đọc hiểu bài này tốt nhất bạn cần:

- Cài đặt sẵn [MÔI TRƯỜNG PHÁT TRIỂN CỦA PYTHON](#).
- Xem qua bài [CÁCH CHẠY CHƯƠNG TRÌNH PYTHON](#).
- Nắm [CÁCH GHI CHÚ](#) và [BIẾN TRONG PYTHON](#).
- [KIỂU DỮ LIỆU SỐ](#), [KIỂU DỮ LIỆU CHUỖI](#) trong Python
- [KIỂU DỮ LIỆU LIST](#), [KIỂU DỮ LIỆU TUPLE](#), [KIỂU DỮ LIỆU SET](#) trong Python.

Bạn và Kteam sẽ cùng tìm hiểu những nội dung sau đây

- Giới thiệu về Dict trong Python
- Cách khởi tạo Dict
- Lấy value trong Dict bằng key
- Thay đổi nội dung Dict trong Python
- Thêm thủ công một phần tử vào Dict

Giới thiệu về Dict trong Python

Dict(Dictionary) cũng là một container như [LIST](#), [TUPLE](#). Có điều khác biệt là những container như List, Tuple **có các index** để phân biệt các phần tử thì Dict dùng các **key** để phân biệt.

Chắc bạn cũng dùng từ điển tiếng Anh để tra từ vựng rồi nhỉ? Có rất nhiều từ vựng trong đó nhưng mà không từ vựng nào giống nhau. Có chăng chúng chỉ giống nhau về nghĩa? Và đó cũng như Dict(Dictionary) hoạt động trong Python

Một Dict gồm các yếu tố sau:

- Được giới hạn bởi cặp ngoặc nhọn {}, tất cả những gì nằm trong đó là những phần tử của Dict.
- Các phần tử của Dict được phân cách nhau ra bởi dấu phẩy (,).
- Các phần tử của Dict phải là một cặp **key-value**
- Cặp **key-value** của phần tử trong Dict được phân cách bởi dấu hai chấm (:)
- Các key buộc phải là một **hash object**

Cách khởi tạo Dict

Sử dụng cặp dấu ngoặc {} và đặt giá trị bên trong

Cú pháp:

```
<key_1: value_1>, <key_2: value_2>, .., <key_n: value_n>
```

Ví dụ:

Python:

```
>>> dic = {'name': 'Kteam', 'member': 69}
>>> dic
{'name': 'Kteam', 'member': 69}
>>> empty_dict = {} # khởi tạo dict rỗng
>>> empty_dict
{}
>>> type(dic) # kiểu dữ liệu dict thuộc lớp 'dict'
<class 'dict'>
```

Sử dụng Dict Comprehension

Điều kiện để sử dụng **dict comprehension** là mỗi giá trị khi được duyệt qua phải bao gồm một giá trị tương ứng với key và một giá trị tương ứng với value.

Ví dụ:

Python:

```
>>> dic = {key: value for key, value in [('name', 'Kteam'), ('member', 69)]}
>>> dic
{'name': 'Kteam', 'member': 69}
```

Sử dụng constructor Dict

Với dict, ta có 4 cách để khởi tạo một Dict bằng constructor:

Khởi tạo một Dict rỗng

Cú pháp:

```
dict()
```

Ví dụ:

Python:

```
>>> dic = dict()
>>> dic
{}
```

Khởi tạo một dict từ một mapping object

Cú pháp:

```
dict(mapping)
```

Trong đó:

Mapping object cũng gần giống so với **dict object**.

- Một object là **Mapping object** khi có đủ hai phương thức **keys** và **__getitem__**.
- **Dict object** cũng là một **mapping object**. Tuy nhiên, không phải mapping object nào cũng là dict object vì dict object không chỉ có hai phương thức keys và **__getitem__** và còn nhiều phương thức khác.

Bạn có thể bỏ qua ví dụ bên dưới hoặc xem để tham khảo vì phần này có thể khá khó hiểu.

Ví dụ:

Python:

```
>>> class Map_Class:
...     def keys(self):
...         return [1, 2, 3]
...     def __getitem__(self, key):
...         return key * 2
...
>>> map_obj = Map_Class()
>>> dic = dict(map_obj)
>>> dic
{1: 2, 2: 4, 3: 6}
```

Khởi tạo bằng iterable

Cú pháp:

dict(iterable)

Trong đó:

iterable này đặc biệt hơn các **iterable** mà bạn dùng để khởi tạo List hay Tuple, đó là các phần tử trong iterable phải có 2 value đó chính là cặp **key-value**.

Bạn có thể dùng List, Tuple hoặc bất cứ container nào (trừ mapping object) để chứa cặp **key-value**.

Ví dụ 1:

Python:

```
>>> iter_ = [('name', 'Kteam'), ('member', 69)]
>>> dic = dict(iter_)
>>> dic
{'name': 'Kteam', 'member': 69}
```

Ví dụ 2:

Python:

```
>>> f = [('ab'), ('cd')]
>>> dict(f)
{'a': 'b', 'c': 'd'}
```

Lưu ý: Nếu các bạn muốn khởi tạo dict bằng chuỗi như trên thì bắt buộc độ dài của chuỗi phải là 2. Giá trị đầu sẽ là key, giá trị thứ 2 là value.

Python:

```
>>> f = [('abc'), ('cde')]
>>> dict(f)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: dictionary update sequence element #0 has length 3; 2 is required
```

Khởi tạo bằng keyword arguments

Cú pháp 1:

dict(kwargs)**

Trong đó:

Bạn chưa tìm hiểu đến hàm, nên khái niệm **keyword arguments** vẫn còn rất xa lạ!

- Cứ hiểu đơn giản là giống như việc bạn khởi tạo biến và giá trị rồi đưa cho dict đó giữ gìn.
- Một lưu ý là những biến này không bị ảnh hưởng hoặc ảnh hưởng gì đến các biến bên ngoài

Ví dụ:

Python:

```
>>> name = 'SpaceX'
>>> member = 696969
>>> dic = dict(name='Kteam', member=69)
>>> dic
{'name': 'Kteam', 'member': 69}
>>> name
'SpaceX'
>>> member
696969
```

Cú pháp 2:

dict(<key_1> = <value_1>, <key_2> = <value_2>, ...)

Khi khởi tạo dict bằng **keyword arguments** chỉ có thể hoạt động khi các **key** là tên các biến, còn các **value** là giá trị của các key tương ứng với nó (các biến có thể được khởi tạo trong cặp ngoặc {}).

Ví dụ:

Python:

```
>>> name = 'SpaceX'
>>> member = 696969
>>> dic = dict(name='Kteam', member=69)
>>> dic
{'name': 'Kteam', 'member': 69}
>>> name
'SpaceX'
>>> member
696969
```

Việc cố gắng khởi tạo **dict** như dưới đây là **không** hợp lệ.

Python:

```
>>> dict((1, 2) = 3)
File "<stdin>", line 1
    dict((1, 2) = 3)
    ^^^^^^^
SyntaxError: expression cannot contain assignment, perhaps you meant "=="?

>>> dict('o' = 3)
File "<stdin>", line 1
    dict('o' = 3)
    ^^^^^
SyntaxError: expression cannot contain assignment, perhaps you meant "=="?
```

Sử dụng Phương thức fromkeys

Cú pháp:

```
dict.fromkeys(iterable, value)
```

Công dụng: Cách này cho phép ta khởi tạo một dict với các **keys** nằm trong một **iterable**. Các giá trị này đều sẽ nhận được một giá trị với mặc định là **None**

Ví dụ:

Python:

```
>>> iter_ = ('name', 'number')
>>> dic_none = dict.fromkeys(iter_)
>>> dic_none
{'name': None, 'number': None}
>>> dic = dict.fromkeys(iter_, 'non None value')
>>> dic
{'name': 'non None value', 'number': 'non None value'}
```

Lấy value trong Dict bằng key

Cú pháp:

```
Your_dict[key]
```

Ví dụ:

Python:

```
>>> dic # ta có một dict như sau
{'name': 'Kteam', 'member': 69}
>>> dic['name']
'Kteam'
>>> dic['member']
69
>>> dic['non_exist'] # hãy chắc chắn rằng key bạn dùng có trong dict
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'non_exist'
```

Thay đổi nội dung Dict trong Python

Dict là một **unhashable object**. Do đó, chắc bạn cũng biết ta có thể thay đổi được nội dung nó hay không. Nếu bạn nào nhanh trí, chắc cũng đã biết được cách thay đổi rồi. Tương tự như List thôi!

Ví dụ:

Python:

```
>>> dic # ta có một dict như sau
{'name': 'Kteam', 'member': 69}
>>> dic['name'] = 'How Kteam'
>>> dic
{'name': 'How Kteam', 'member': 69}
>>> dic['member'] = dic['member'] + 1
>>> dic
{'name': 'How Kteam', 'member': 70}
```

Thêm thủ công một phần tử vào dict

Cách này khá giống với cách bạn thay đổi nội dung của Dict. Khác ở chỗ, bây giờ bạn sẽ sử dụng một **key** chưa hề có trong dict.

Ví dụ:

Python:

```
>>> dic # ta có một dict như sau
{'name': 'Kteam', 'member': 69}
>>> dic['slogan'] = 'Free Education'
>>> dic
{'name': 'Kteam', 'member': 69, 'slogan': 'Free Education'}
```

Vấn đề cần lưu tâm khi sử dụng dict

Các bạn còn nhớ phần "**Vấn đề cần lưu tâm khi sử dụng list**" chứ? Đối với dict, ta cũng cần có các lưu ý tương tự như vậy. Kteam sẽ đưa ra một vài ví dụ để cho các bạn hiểu. Nếu cần thiết, các bạn có thể xem lại bài "[Kiểu dữ liệu list trong Python – Phần 1](#)".

Các lỗi dễ xảy ra với list cũng có thể xảy ra với dict

Python:

```
>>> a = {1: '1', 2: '2', 3: '3'}
>>> b = a
>>> a is b
True
>>> b[1] = 't'
>>> a
{1: 't', 2: '2', 3: '3'}
>>> b
{1: 't', 2: '2', 3: '3'}
>>> a[4] = '4'
>>> b
{1: 't', 2: '2', 3: '3', 4: '4'}
>>> a
{1: 't', 2: '2', 3: '3', 4: '4'}
```

Và tất nhiên, có lỗi thì sẽ có cách sửa chữa

Python:

```
>>> a = {1: '1', 2: '2', 3: '3'}
>>> b = a.copy() # Phương thức copy
>>> b is a
False
>>> b
{1: '1', 2: '2', 3: '3'}
>>> c = dict(a) # Khởi tạo dict mới
>>> c is a
False
>>> c
{1: '1', 2: '2', 3: '3'}
```

Củng cố bài học

Đáp án bài trước

Bạn có thể tìm thấy câu hỏi của phần này tại CÂU HỎI CÙNG CỐ trong bài [KIỂU DỮ LIỆU SET TRONG PYTHON](#).

Ví khi:

Python:

```
>>> a = {1, 2}
>>> b = a
```

Ta đã cho a và b cùng trỏ vào một chỗ. Do đó thay đổi b thì a cũng sẽ bị tác động.

Muốn giải quyết chuyện này ta nên sử dụng phương thức copy

Python:

```
>>> a = {1, 2}
>>> b = a.copy()
>>> b.clear()
>>> b
set()
>>> a
{1, 2}
```

Kết luận

Bài viết này đã giới thiệu cho các bạn cơ bản về KIỂU DỮ LIỆU DICT TRONG PYTHON.

Ở bài sau, Kteam sẽ đề cập về các phương thức của [KIỂU DỮ LIỆU DICT - Phần 2](#)

Cảm ơn bạn đã theo dõi bài viết. Hãy để lại bình luận hoặc góp ý của mình để phát triển bài viết tốt hơn. Đừng quên **"Luyện tập – Thử thách – Không ngại khó"**.